1)ENERGY MANAGEMENT SYSTEM:

Develop a Java program to create an abstract class EnergySource with members Source ID, Source Name, and Energy Generated (in kWh). Include an abstract method calculateEfficiency(). Derive the classes SolarEnergy and WindEnergy from EnergySource and implement the method based on the following criteria: For SolarEnergy, efficiency is calculated as (Energy Generated / 5000) × 100. For WindEnergy, efficiency is calculated as (Energy Generated / 8000) × 100. Display source details and efficiency percentage. Demonstrate dynamic method dispatch using an EnergySource reference.

```
abstract class EnergySource {
    String sourceId;
    String sourceName;
    double energyGenerated;

    EnergySource(String sourceId, String sourceName, double
energyGenerated) {
        this.sourceId = sourceId;
        this.sourceName = sourceName;
        this.energyGenerated = energyGenerated;
    }


    abstract double calculateEfficiency();


    void displayDetails() {
        System.out.println("Source ID: " + sourceId);
        System.out.println("Source Name: " + sourceName);
        System.out.println("Energy Generated: " +
energyGenerated + " kWh");
        System.out.printf("Efficiency: %.2f\n",
calculateEfficiency());
        System.out.println("----------------------------------
-");
    }
}


class SolarEnergy extends EnergySource {

    SolarEnergy(String sourceId, String sourceName, double
```

```
energyGenerated) {
        super(sourceId, sourceName, energyGenerated);
    }


    double calculateEfficiency() {
        return (energyGenerated / 5000.0) * 100.0;
    }
}


class WindEnergy extends EnergySource {

    WindEnergy(String sourceId, String sourceName, double
energyGenerated) {
        super(sourceId, sourceName, energyGenerated);
    }

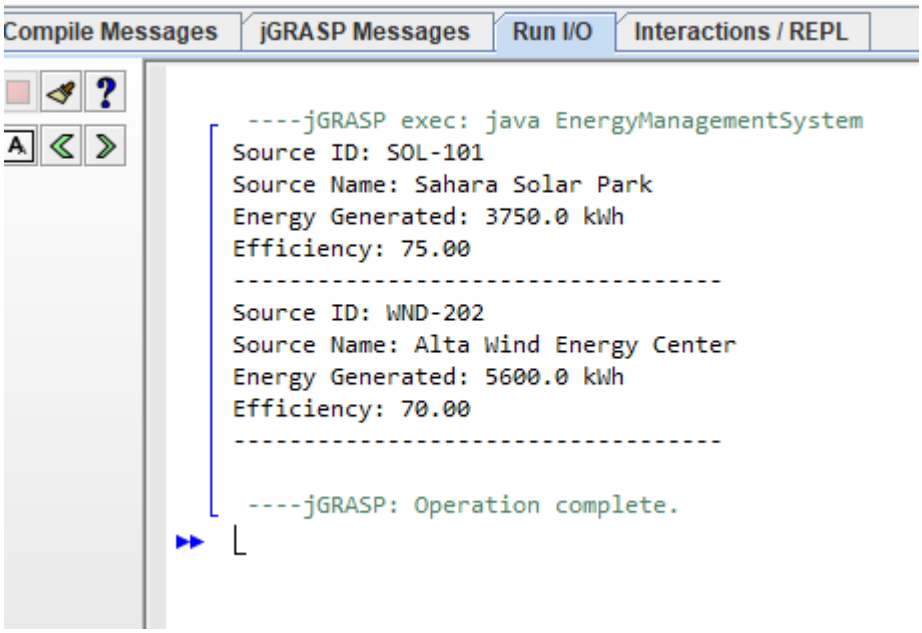
    double calculateEfficiency() {
        return (energyGenerated / 8000.0) * 100.0;
    }
}


public class EnergyManagementSystem {
    public static void main(String[] args) {

        EnergySource ref;

        ref = new SolarEnergy("SOL-101", "Sahara Solar Park",
3750.0);
        ref.displayDetails();


        ref = new WindEnergy("WND-202", "Alta Wind Energy
Center", 5600.0);
        ref.displayDetails();
    }
}
```




PARKING MANAGEMENT SYSTEM:

Develop a Java program to create an interface ParkingCharge containing the methods calculateCharge() and displayParkingDetails(). Create a class VehicleParking with members Vehicle Number, Vehicle Type, and Parking Hours. Derive the classes TwoWheeler and FourWheeler from VehicleParking and implement the interface. Calculate the parking charge as follows: For TwoWheeler, charge Rs. 20 per hour for the first two hours and Rs. 10 per hour for each additional hour. For FourWheeler, charge Rs. 50 per hour for the first two hours and Rs. 30 per hour for each additional hour. Display vehicle details and total parking charge. Demonstrate runtime polymorphism using a ParkingCharge reference.

```
interface ParkingCharge {
    double calculateCharge();
    void displayParkingDetails();
}

class VehicleParking {
    String vehicleNumber;
    String vehicleType;
    int parkingHours;

    public VehicleParking(String vehicleNumber, String vehicleType, int
parkingHours) {
```

```
        this.vehicleNumber = vehicleNumber;
        this.vehicleType = vehicleType;
        this.parkingHours = parkingHours;
    }
}

class TwoWheeler extends VehicleParking implements ParkingCharge {

    public TwoWheeler(String vehicleNumber, int parkingHours) {
        super(vehicleNumber, "Two Wheeler", parkingHours);
    }

   public double calculateCharge() {
        if (parkingHours <= 2) {
            return parkingHours * 20.0;
        } else {
            return (2 * 20.0) + ((parkingHours - 2) * 10.0);
        }
    }

   public void displayParkingDetails() {
        System.out.println("Vehicle Number: " + vehicleNumber);
        System.out.println("Vehicle Type: " + vehicleType);
        System.out.println("Parking Hours: " + parkingHours + " hrs");
        System.out.println("Total Parking Charge: Rs. " +
calculateCharge());
        System.out.println("-----------------------------------");
    }
}

class FourWheeler extends VehicleParking implements ParkingCharge {

    public FourWheeler(String vehicleNumber, int parkingHours) {
        super(vehicleNumber, "Four Wheeler", parkingHours);
    }

   public double calculateCharge() {
        if (parkingHours <= 2) {
            return parkingHours * 50.0;
        } else {
            return (2 * 50.0) + ((parkingHours - 2) * 30.0);
        }
    }

    public void displayParkingDetails() {
        System.out.println("Vehicle Number: " + vehicleNumber);
        System.out.println("Vehicle Type: " + vehicleType);
        System.out.println("Parking Hours: " + parkingHours + " hrs");
        System.out.println("Total Parking Charge: Rs. " +
calculateCharge());
        System.out.println("-----------------------------------");
    }
}

public class ParkingManagementSystem {
    public static void main(String[] args) {

        ParkingCharge parkingRef;

        parkingRef = new TwoWheeler("KA-01-AB-1234", 5);
        parkingRef.displayParkingDetails();
```

```
        parkingRef = new FourWheeler("KA-05-CD-5678", 4);
        parkingRef.displayParkingDetails();
    }
}
```

OUTPUT:

